JOB PORTAL

A PROJECT REPORT

Submitted by

Disha (24BCS10773)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE & ENGINEERING

Chandigarh University

APRIL 2026

1. Introduction and Project Overview

1.1 What is EasyJobs?

EasyJobs is a full-stack web-based job portal developed as part of the Full Stack Development course for the academic year 2025–26. The application serves as a bridge between job seekers and employers by providing a single, centralized platform where employers can post job openings and candidates can discover, search, and apply for relevant positions.

The portal is built using modern industry-standard technologies — React 18 on the frontend and Spring Boot 3.2 on the backend — connected through a RESTful JSON API. All user data, job listings, and applications are persisted using an H2 file-based database. User authentication is handled securely using JSON Web Tokens (JWT), ensuring that protected operations are accessible only to authorized users.

1.2 Purpose and Motivation

Traditional job hunting involves visiting multiple websites, filling out repetitive forms, and losing track of applications. EasyJobs solves this by:

- Giving employers a simple interface to publish job listings instantly.
- Allowing job seekers to search and filter openings by keyword and location.
- Letting applicants submit their details and cover letter directly through the portal.
- Providing a dashboard where users can track the status of every application they submitted.

1.2 Purpose and Motivation

Traditional job hunting involves visiting multiple websites, filling out repetitive forms, and losing track of applications. EasyJobs solves this by:

- Giving employers a simple interface to publish job listings instantly.
- Allowing job seekers to search and filter openings by keyword and location.
- Letting applicants submit their details and cover letter directly through the portal.
- Providing a dashboard where users can track the status of every application they submitted.

1.3 Technology Stack

Layer	Technology Used
Frontend	React 18, React Router v6, Axios, Vite
Backend	Spring Boot 3.2, Spring Security, Spring Data JPA
Database	H2 (file-based, persistent — easyjobsdb.mv.db),MYSQL

Layer	Technology Used
Authentication	JWT via jjwt 0.11.5, BCrypt password hashing
Build Tools	Maven (backend), Vite (frontend)
Languages	Java 17 (backend), JavaScript / JSX (frontend)

1.4 Project Structure

The project is divided into two top-level modules:

Backend — Spring Boot REST API

- controller/ — AuthController, JobController
- model/ — User, Job, Application (JPA entities)
- repository/ — UserRepository, JobRepository, ApplicationRepository
- service/ — AuthService, JobService (business logic)
- security/ — JwtFilter (request interceptor), JwtUtil (token generation/validation)
- config/ — SecurityConfig (CORS, route permissions), DataSeeder (sample data)

Frontend — React Single Page Application

- pages/ — Home, Jobs, JobDetail, Login, Register, PostJob, MyApplications
- components/ — Navbar, JobCard (reusable UI components)
- context/ — AuthContext (global auth state via React Context API)
- services/ — api.js (Axios instance with JWT interceptor)

1.5 How the System Works

When a user opens the application in their browser, the React frontend loads and renders the UI. API calls are made to the Spring Boot server running on port 8080. The server validates the request (checking the JWT token if the route is protected), performs the required business logic through the service layer, and reads/writes data via JPA repositories to the H2 database. The response is returned as JSON and displayed in the browser.

Roles are assigned at registration. A USER (job seeker) can browse jobs, apply, and track applications. An EMPLOYER can do everything a USER can, plus post new job listings.

2. Objectives

1. Provide a centralized platform where employers can post jobs and job seekers can discover opportunities in one place.
2. Implement secure authentication using JWT-based login and registration with role-based access control (USER / EMPLOYER).
3. Demonstrate complete full-stack integration — a React frontend communicating with a Spring Boot REST API backed by a persistent database.
4. Track application status — applicants can view all submitted applications and their 4. current status (PENDING / REVIEWED / ACCEPTED / REJECTED).
5. Allow job seekers to apply directly through the portal by submitting personal details and a cover letter.
6. Enable job search and filtering by keyword and location so users can find relevant listings quickly.
7.

3. Functional Requirements

3.1 User Authentication

Requirement
Users must be able to register with their name, email, password, and role (USER or EMPLOYER).
Registered users must be able to log in using their email and password.
On successful login, the system must return a JWT token valid for 24 hours.
The JWT token must be automatically attached to all subsequent protected API requests via an Axios interceptor.
Passwords must be stored in hashed form using BCrypt — plain-text storage is not permitted.

3.2 Job Listings

Requirement
Any visitor (unauthenticated) must be able to browse all job listings.
Users must be able to search jobs by keyword (title or company name) and by location.
Each listing must display: title, company, location, salary, job type, description, requirements, and posted date.
Authenticated employers must be able to post new job listings through a dedicated form.
All job listings must be stored persistently in the database and survive server restarts.

3.3 Job Applications

Requirement
Authenticated users (job seekers) must be able to apply for any job listing.
The application form must collect: applicant name, email, phone number, and cover letter.
Each application must be linked in the database to both the applying user and the target job.
Application status must default to PENDING on submission.
Logged-in users must be able to view a list of all applications they have submitted.
Employers must be able to view all applications received for a specific job they posted.

3.4 Navigation and Pages

Requirement
The app must have a persistent Navbar that shows different links depending on whether the user is logged in.
The Home page must display a hero section with a call-to-action button to browse jobs.
The Jobs page must list all available jobs with working keyword and location search controls.
Clicking a job card must navigate to a Job Detail page displaying full information and an Apply button.

4. Flow Diagrams

4.1 User Registration and Login Flow

```
      [Start]
        |
        v
  [User visits website]
        |
        v
    [Click Register/Login]
        |
        v
     [Enter Details]
(Name, Email, Password, Role)
        |
        v
 [Send POST /api/auth/register]
        |
        v
     [Validation Check]
       /       \
     Yes       No
     |          |
```

```
     v          v
[Save User]   [Show Error]

[BCrypt  Hash]      |
     |           |
     v           |
[Generate JWT Token]
     |
     v
[Store Token in LocalStorage]
     |
     v
 [Redirect to Home/Jobs]
     |
     v
   [End]
```

4.2 Job Search and Apply Flow

```
      [Start]
         |
         v
    [Open Jobs Page]
         |
         v
 [GET /api/jobs (Fetch Jobs)]
         |
         v
   [Display Job Listings]
         |
         v
    [Click Job Card]
         |
         v
 [GET /api/jobs/{id}]
         |
         v
   [Show Job Details]
         |
         v
    [Click Apply Now]
         |
         v
     [Login Check]
       /     \
```

```
    Yes      No
    |        |
    v        v
[Show Form]  [Redirect Login]
    |
    v
[Fill Application Details]
    |
    v
[POST /apply (JWT Token)]
    |
    v
[Save Application (PENDING)]
    |
    v
 [Show Success Message]
    |
    v
  [End]
```

4.3 Employer Post Job Flow

```
        [Start]
          |
          v
    [Employer Login]
          |
          v
   [Go to Post Job Page]
          |
          v
    [Fill Job Details]
          |
          v
 [POST /api/jobs (JWT Token)]
          |
          v
     [Save Job]
          |
          v
 [Job Visible on Jobs Page]
          |
          v
        [End]
```

4.4 System Architecture Flow

```
[Frontend (React - Port 3000)]
            |
             HTTP Requests (JSON)
            v
 [Backend (Spring Boot - Port 8080)]
            |
            v
      [JWT Authentication Filter]
            |
     +-------+-------+
     |             |
     v             v
[AuthController]  [JobController]
     |             |
     v             v
[AuthService]    [JobService]
     |             |
     v             v
[UserRepository] [JobRepository]
              [ApplicationRepository]
            |
            v
     [H2 Database]
```

5. Page Layout

The frontend consists of the following pages and components. Each page is a React component rendered by React Router v6 based on the current URL.

5.1 Navbar (Persistent across all pages)

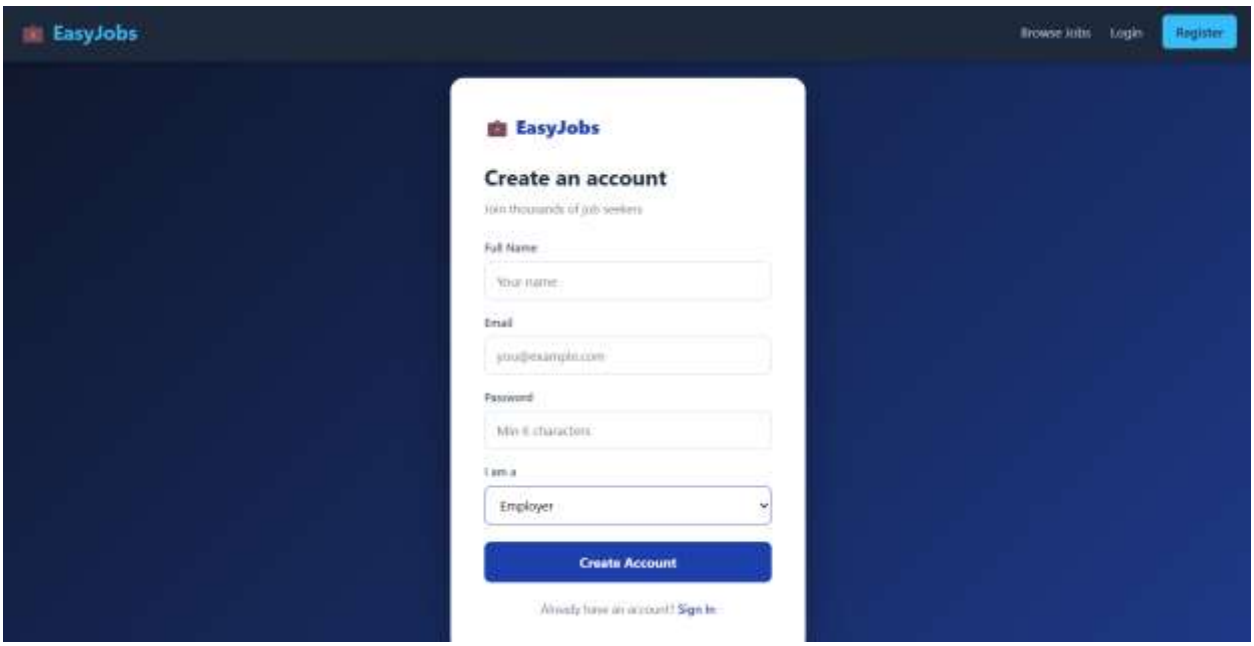
- Left side: EasyJobs logo / brand name
- Center links: Home | Jobs | Post Job (visible only to logged-in employers)
- Right side (guest): Login button | Register button
- Right side (logged in): My Applications | Logout button

5.2 Home Page (/)

- Hero section with a bold headline and a 'Browse Jobs' call-to-action button
- Brief description of what EasyJobs offers to job seekers and employers

5.3 Jobs Page (/jobs)

- Keyword search input and location filter input at the top
- Grid of JobCard components — each card shows: Job Title, Company, Location, Salary, Job Type, Posted Date
- Clicking any card navigates to that job's detail page



5.4 Job Detail Page (/jobs/:id)

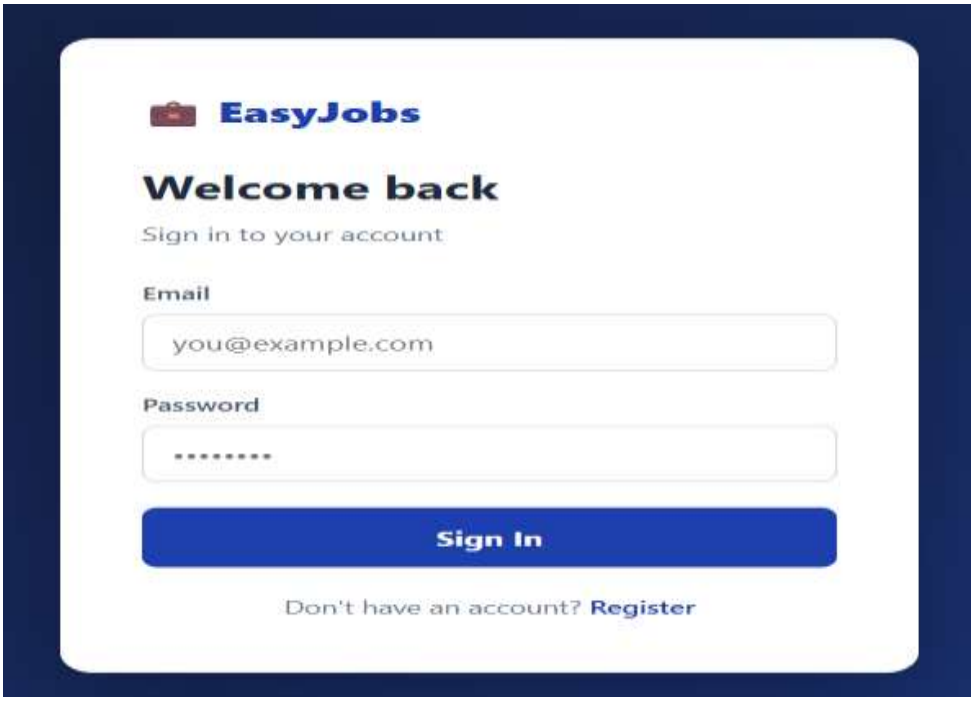
- Displays full job information: title, company, location, salary, type, description, requirements
- 'Apply Now' button — if not logged in, redirects to /login
- Application form fields: Name, Email, Phone Number, Cover Letter, Submit button

5.5 Post Job Page (/post-job)

- Form fields: Title, Company, Location, Salary, Job Type (Full-Time / Part-Time / Remote), Description, Requirements
- Submit triggers POST /api/jobs with JWT in the Authorization header

5.6 Login Page (/login)

- Email input and Password input
- Submit calls POST /api/auth/login and stores the returned JWT in localStorage
- On success, user is redirected to the Jobs page

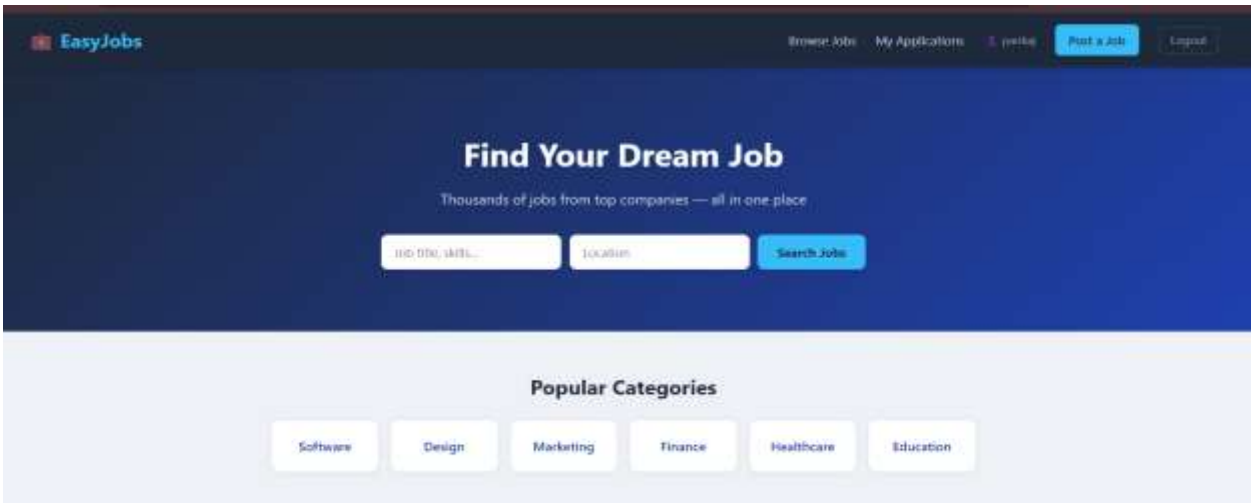


5.7 Register Page (/register)

- Fields: Full Name, Email, Password, Role selector (USER / EMPLOYER)
- Submit calls POST /api/auth/register

5.8 My Applications Page (/my-applications)

- Table listing all applications submitted by the currently logged-in user
- Columns: Job Title | Company | Applied Date | Status
- Status values: PENDING | REVIEWED | ACCEPTED | REJECTED



6. Code Snippets

- Database Mapping: It maps the Application class to a database table.
- Automatic ID: It uses @Id and @GeneratedValue to automatically create a unique, incrementing ID for every new entry.

```java
public class Application {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String applicantName;
    private String email;
    private String phone;

    @Column(length = 2000)
    private String coverLetter;

    private String status = "PENDING"; // PENDING, REVIEWED, ACCEPTED, REJECTED
```

- CSS for designing

```css
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Inter', 'Segoe UI', sans-serif;
  background-color: #f1f5f9;
  color: #1e293b;
  line-height: 1.6;
}

a {
  text-decoration: none;
}
```

- Validation: Uses @NotBlank on the title and company fields. This ensures that a job cannot be saved to the database if the title or company name is empty or just whitespace.
- Data Structure: It defines the basic attributes needed for a job listing—specifically a unique identifier, a job title, and the company name.

```java
@Entity
@Table(name = "jobs")
public class Job {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    private String title;

    @NotBlank
    private String company;
```

7 Limitations

- Limited Security Features
- Basic User Interface
- Limited Job Filtering
- No Notification System
- No Real-Time Updates

8. Future Enhancements

8.1 Advanced Security Features

- Implement Multi-Factor Authentication (MFA) for secure login
- Add Email verification during registration
- Use CAPTCHA to prevent bot registrations
- Improve JWT handling with refresh tokens

8.2 Resume Upload and Profile Management

- Allow users to upload resumes (PDF/DOC format)
- Provide a complete user profile section with:
    - Skills
    - Education
    - Experience
- Enable employers to view detailed candidate profiles

8.3 Advanced Search and Filtering

- Add filters such as:
- Salary range
- Experience level
- Job type (Internship, Full-time, Part-time)
- Implement sorting options (latest jobs, highest salary, etc.)

8.4 Notification System

- Integrate email notifications for:
    - Job application status
    - New job postings
- Add in-app notifications for better user engagement

9. Conclusion

The Job Application Portal developed in this project successfully demonstrates the implementation of a full-stack web application using modern technologies such as React for frontend and Spring Boot for backend.

The system provides essential functionalities including:

- User registration and authentication
- Job posting by employers
- Job searching and application by job seekers

The use of REST APIs and JWT-based authentication ensures smooth communication between frontend and backend while maintaining basic security.

Although the system is functional and efficient for small-scale use, it has certain limitations in terms of scalability, advanced features, and user experience. These limitations provide opportunities for future enhancements and further development.

Overall, the project helped in understanding:

- Full-stack development concepts
- Client-server architecture
- Database integration
- Real-world application design

This system can be further extended into a complete industry-level job portal with additional features and improvements.